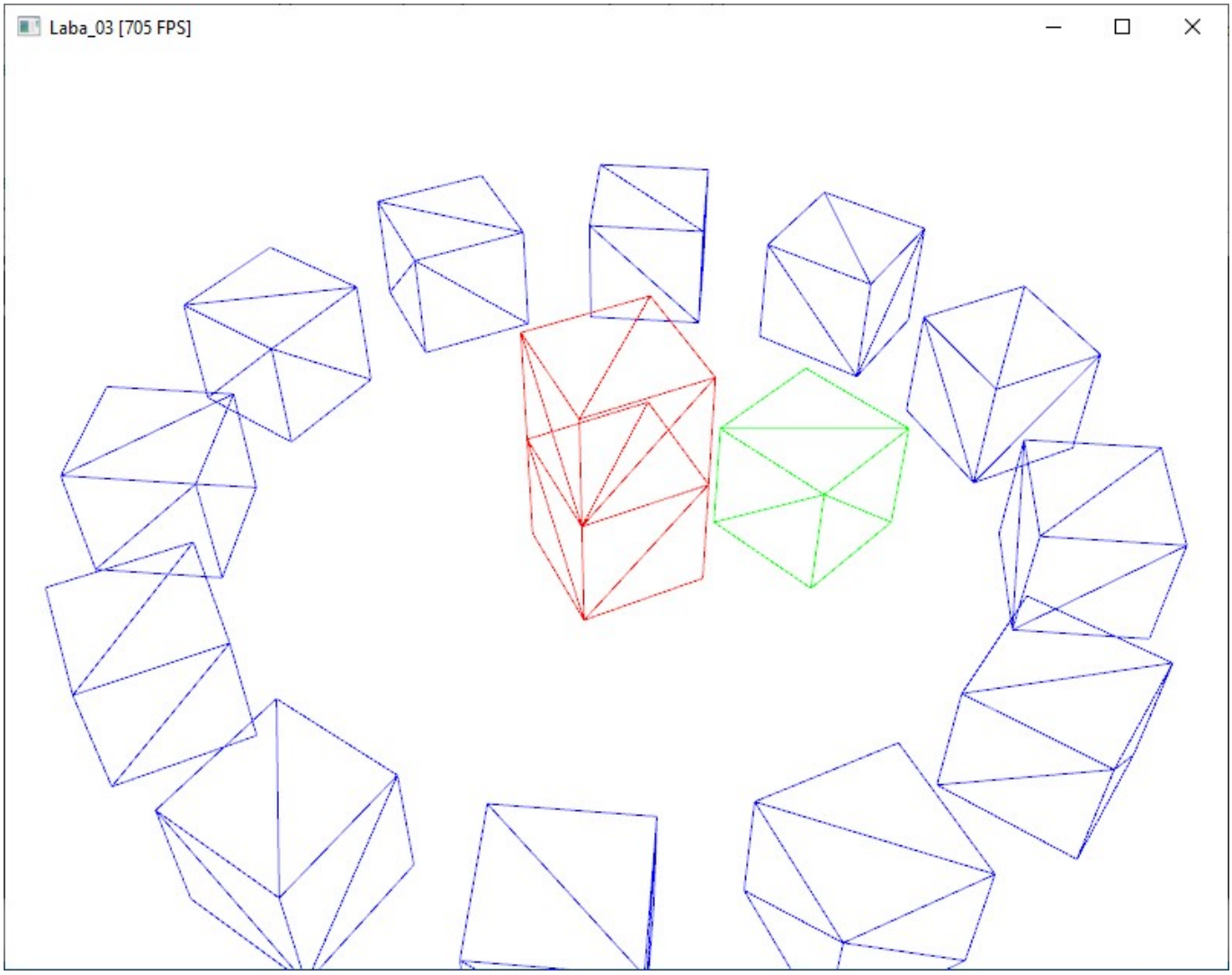
# Лабораторная работа №03

## Преобразование вершин с использованием GLSL шейдеров, матриц наблюдения, модели и проекции.

В лабораторной работе №2 необходимо разместить на сцене нескольких графических объектов, представленных кубиками, а также реализовать перемещение направленной камеры так, чтобы сцену можно было рассмотреть со всех сторон.

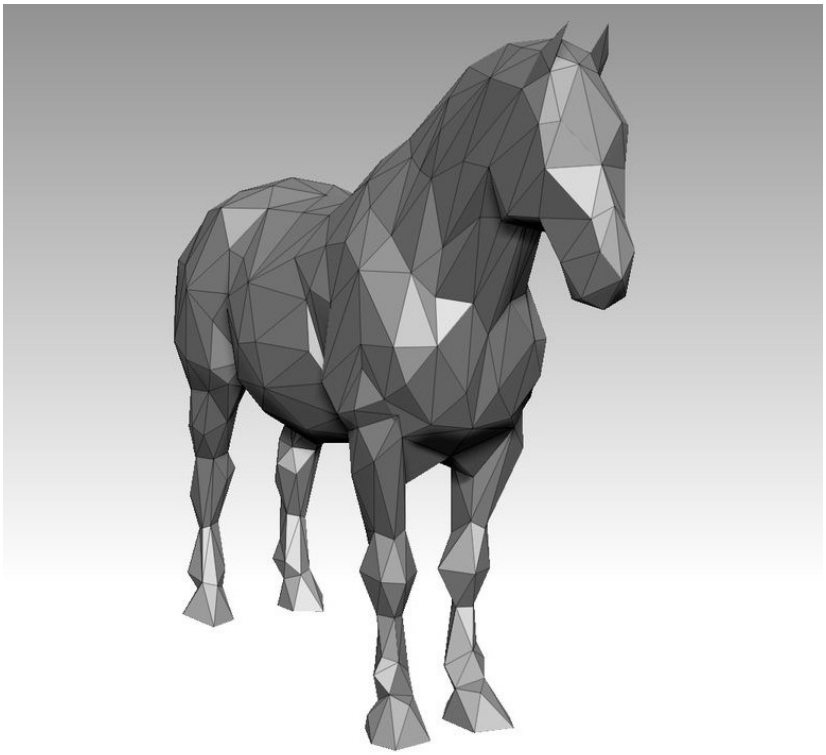
Для реализации поставленной задачи необходимо использовать вершинный шейдер, который обеспечивает преобразование вершин с учетом матриц проекции, наблюдения и модели. Необходимые матрицы генерируются в основной программе и передаются в GLSL-шейдер посредством uniform-переменных, работа с которыми была рассмотрена в предыдущей лабораторной.

В процессе реализации программы необходимо написать класс GraphicObject, для удобной работы с графическими объектами, а также класс Camera, для работы с камерой. Для непосредственного вывода полигонов на экран требуется использовать функцию drawBox(), полный текст который приводится в приложении к данным методическим материалам.



# Процесс преобразования вершин.

Трехмерные модели, выводимые на экран, как правило, состоят из полигонов (чаще всего – треугольников), каждый из которых описывается своими вершинами. Вершины, в свою очередь, описываются всеми своими атрибутами, важнейшим из которых являются геометрическая координата, задающая положение вершины в пространстве.



Процесс вывода моделей на экран подразумевает определение того, где будет располагаться каждая вершина и, как правило, выполняется в соответствии со следующей последовательностью действий:

1. Прежде всего, необходимо выбрать шейдер, который будет использоваться для вывода модели на экран. Именно вершинный шейдер определяет алгоритм, который будет использован для вычисления координаты вершины.

2. Далее следует установить в выбранном шейдере значение uniform-переменных, влияющих на процесс преобразования вершин. Например, в предыдущей лабораторной работе использовалась переменная “uniform vec2 offset” для вычисления позиции вершины. В рамках данной лабораторной работы для вычисления позиции вершины будут использоваться матрица проекции, матрица наблюдения и матрица модели которые также являются uniform-переменными.

3. Затем необходимо непосредственно передать в OpenGL все атрибуты всех вершин, то есть вызвать команды для передачи необходимых данных в видеокарту (так называемые draw-call’ы). Данные команды будут рассмотрены в следующей лабораторной работе, здесь же для вывода кубика на экран будет использоваться функция drawBox, полный текст которой приведен в приложении. Данная функция генерирует и передает в OpenGL трехмерные геометрические координаты вершин куба. Сам куб лежит в начале локальной системы координат и имеет длину ребер равную единице, то есть все координаты лежат в пределах от -0.5 до +0.5 по каждой оси (x, y, z).

4. На следующем этапе выполняется вершинный шейдер. Вершинный шейдер, используя uniform-переменные, которые являются общими для всех вершин данной модели, и attribute-переменные, которые являются индивидуальными для каждой вершины, должен вычислить преобразованные координаты вершины и записать их во встроенную переменную gl_Position.

Переменная gl_Position объявлена как вектор из четырех компонент (x, y, z, w). Важную роль несут не только привычные трехмерные координаты (x, y, z), но и w-компонента, поскольку именно благодаря ей можно добиться эффекта перспективы, при котором объекты, расположенные дальше от наблюдателя, кажутся меньше.

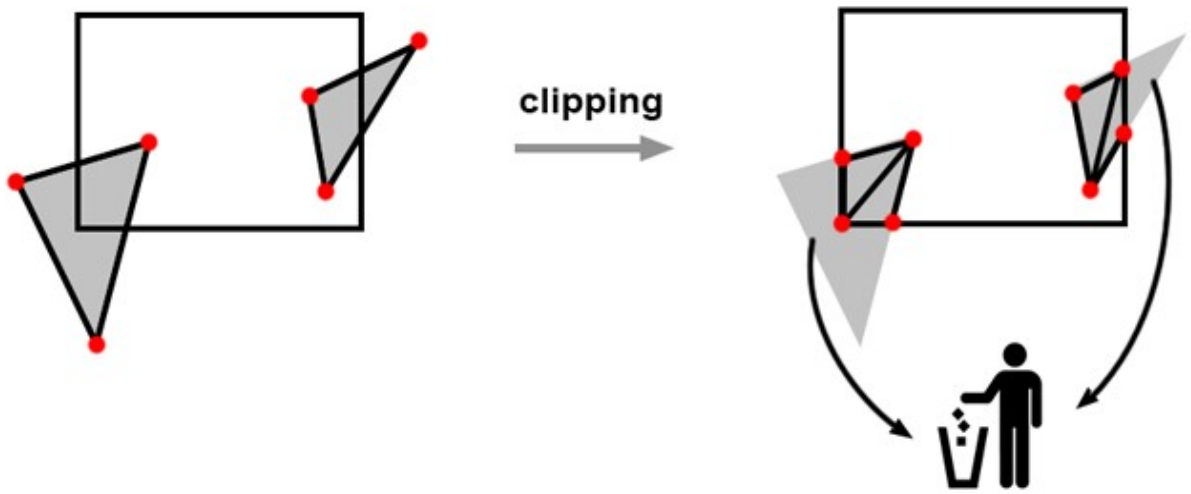
Вычисленное значение называется усеченной координатой (clip space) и передается дальше по конвейеру рендеринга. На этом работа вершинного шейдера закончена.

5. Далее идет шаг, называемый отсечением (clipping).

Усеченные координаты используются не только для определения того, где лежит вершина, но и для определения того, видна ли вершина на экране, поскольку некоторые объекты или их отдельные вершины могут лежать вне области видимости. В частности, вершина лежит в области видимости и попадает на экран только в том случае, если выполняются следующие условия:

$$-w_c \leq x_c \leq w_c$$
$$-w_c \leq y_c \leq w_c$$
$$-w_c \leq z_c \leq w_c$$

В случае, если какой-либо полигон попадает на экран лишь частично, OpenGL добавляет новые вершины. Данный процесс происходит автоматически и не нуждается в дополнительной настройке или участии программиста:




6. Затем следует шаг, который называется перспективным делением.

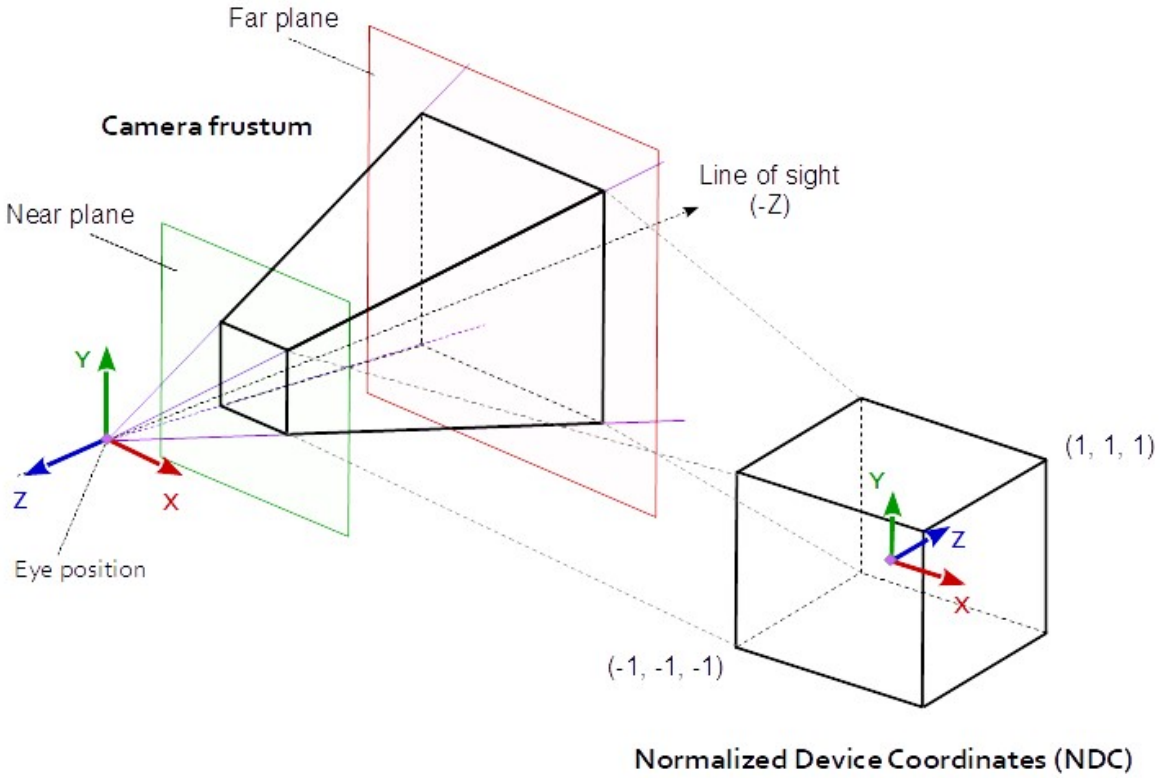
Для всех вершин, полученных после предыдущего шага, усеченная координата делится на свою компоненту w, в результате чего получается вектор (x/w, y/w, z/w, 1). Полученное значение называется нормализованными координатами устройства (normalized device coordinates, NDC). Именно это значение определяет, в каком месте экрана будет лежать вершина. На экране будут видны только те вершины, которые умещаются в «нормализованный куб» - куб в котором компоненты x, y и z меняются в пределах [-1; +1].

Например, вершина, чья x-координата равны «-1» будет лежать на левой границе области вывода. Вершина, чья x-координата равна «+1» будет лежать на правой границе области вывода.

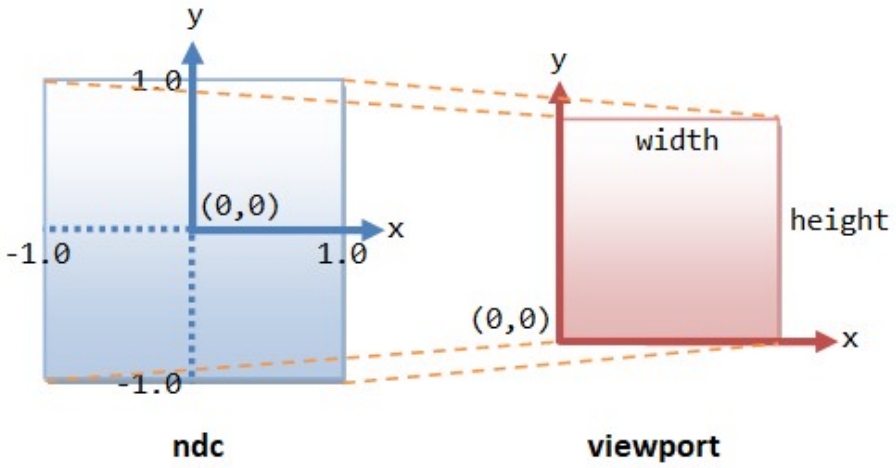
Координата y определяет положение на нижней («-1») или верхней («+1») области вывода.

Значение Z, хоть и не влияет непосредственно на положение вершины, но используется для принятия решения о видимости/невидимости вершины и связанных с ней фрагментов. В частности, вершина не будет видна вообще, если её Z-координата лежит вне диапазона [-1; +1]. Кроме того, z-координата участвует в определении видимости фрагментов с помощью теста глубины (z-теста).

Зачастую, в трехмерной компьютерной графике область видимости определяется с помощью усеченной пирамиды видимости (frustum). Рисунок ниже наглядно показывает соответствие между усеченной пирамидой видимости и системой нормализованных координат устройства. Процесс перехода от усеченной пирамиды видимости к «нормализованному» кубу можно представить как «растяжение» или «сжатие» трехмерного пространства:




7. Последним этапом является трансформация порта просмотра. Трансформация порта просмотра задается с помощью функции glViewport и указывает, как абстрактные нормализованные координаты устройства преобразуются в реальные пиксели на экране. В частности, данная функция позволяет указать область окна, в которую будет осуществляться вывод изображения.



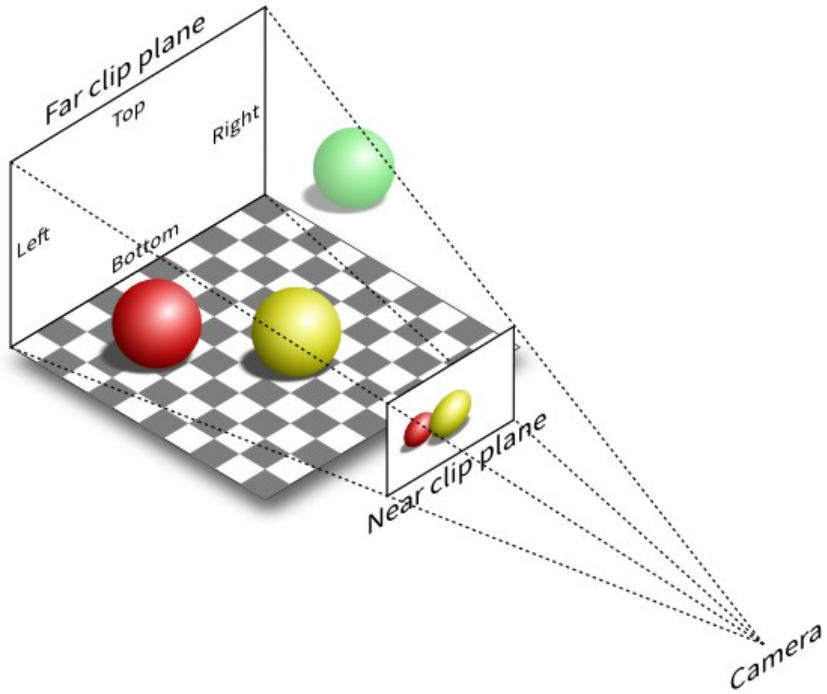

Приведенная выше последовательность действий выполняется всегда. При этом следует отметить, что непосредственно преобразование вершин, то есть вычисление встроенной переменной gl_Position, может осуществляться произвольным образом. На практике, тем не менее, обычно используют стандартный подход, основанный на матрицах. В частности, в рамках данной лабораторной работы, для преобразования вершин необходимо использовать матрицу проекции, матрицу наблюдения и матрицу модели точно также, как они использовались в фиксированном конвейере рендеринга в лабораторных работах предыдущего семестра.

# Использование матриц преобразования (трансформации) для размещения объектов сцены и установки параметров камеры.

Традиционно преобразование вершин для вывода моделей на экран выполняется с использованием трех матриц: матрицы модели, матрицы наблюдения и матрицы проекции. Далее каждая из этих матриц будет рассмотрена независимо, а также будут приведены участки кода, демонстрирующие примеры формирования соответствующих матриц.

***Матрица проекции.***

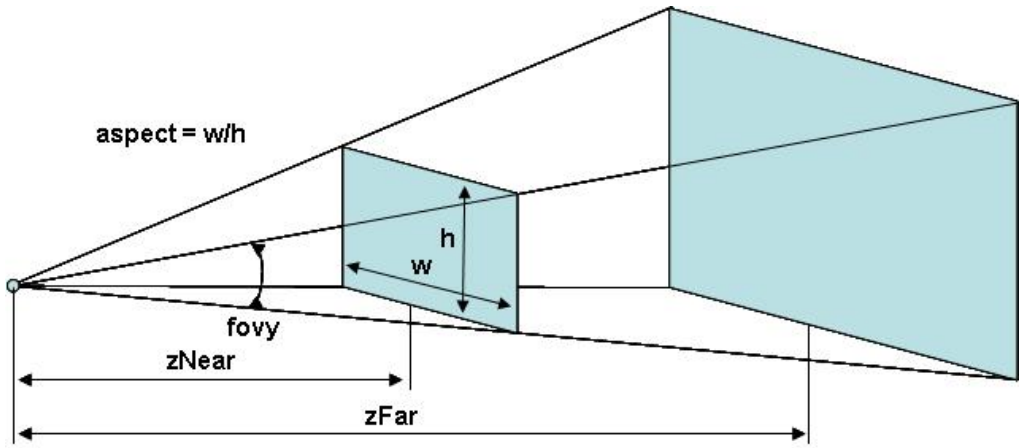
Матрица проекции задает способ проецирования трехмерных координат на экран и одновременно устанавливает область пространства, видимого наблюдателю. Все что лежит вне области видимости не будет выведено на экран. Все что лежит внутри области видимости отображается на экране. Часто для вывода трехмерных объектов используется перспективная проекция и соответствующая ей усеченная пирамида видимости (frustum). В дальнейшем, после выполнения перспективного деления, полученная усеченная пирамида преобразуется к «нормализованному» кубу. Таким образом, ближняя часть пространства как бы растягивается, и именно поэтому объекты, лежащие дальше от наблюдателя, на экране занимают меньшую площадь при тех же размерах:




В проекте предыдущего семестра, при использовании фиксированного конвейера рендеринга OpenGL, можно было использовать различные встроенные функции, например, glFrustum или gluPerspective для генерации матрицы проекции. В современной версии OpenGL, при использовании шейдеров, такого понятия как текущая матрица проекции или текущая матрица наблюдения модели не существует, поэтому данные функции не поддерживаются, и матрицы необходимо генерировать самостоятельно.

Для удобства генерации стандартных матриц проекции библиотека glm предлагает функцию perspective, которая является аналогом функции gluPerspective. Данная функция имеет четыре параметра, которых вполне достаточно для определения области видимости:

- fovy – угол обзора в вертикальном направлении (в радианах);
- aspect – отношение ширины окна (w) к его высоте (h);
- zNear – ближняя плоскость отсечения;
- zFar – дальняя плоскость отсечения.




Ниже приведен код для генерации матрицы проекции. Данная последовательность инструкций генерирует матрицу проекции так, чтобы описать объем видимости имеющий угол охвата по вертикали в 35 градусов, ближнюю плоскость отсечения равную 1.0 и дальнюю равную 100.0. Таким образом, на экран будут выводиться только те объекты, которые находятся от наблюдателя не менее чем в одной и не более чем в 100 единицах. Аспект установлен в вышеуказанное значение, поскольку по умолчанию размер окна в примере равен 800 на 600 пикселей. В случае изменения размеров окна матрица проекции должна быть переопределена заново:

```
// формируем матрицу проекции
mat4 projectionMatrix;
projectionMatrix = perspective(radians(35.0), 800.0 / 600.0, 1.0, 100.0);
```

Отдельно следует отметить важность того, чтобы аспект порта просмотра, устанавливаемый функцией glViewPort, и аспект матрицы проекции совпадали. Это необходимо для того, чтобы избежать искажений объектов, при которых круг будет выведен как эллипс, а квадрат как прямоугольник. Следовательно, матрицу проекции следует переопределять каждый раз, когда меняется размер окна или порт просмотра.

***Матрица вида (матрица камеры, матрица наблюдателя).***

Матрица вида необходима для преобразования координат из глобальной системы координат в систему координат наблюдателя (view space). Матрица вида строится с учетом того, где находится наблюдатель, и в какую точку направлен его взгляд. Для облегчения генерирования матрицы наблюдения в библиотеке glm если функция lookAt, которая является аналогом ранее используемой функции gluLookAt. Пример кода для генерации матрицы камеры приведен ниже:

```
// генерирование матрицы камеры
mat4 viewMatrix;
// позиция камеры - (0, 3, 5)
vec3 eye = vec3(0.0, 3.0, 5.0);
// точка, в которую направлена камера - (0, 0, 0);
vec3 center = vec3(0, 0, 0);
// примерный вектор "вверх" (0, 1, 0)
vec3 up = vec3(0, 1, 0);
// матрица камеры
viewMatrix = lookAt(eye, center, up);
```

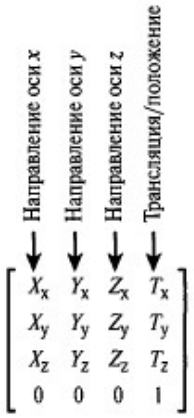
Следует отметить, что в данной лабораторной работе требуется не только сгенерировать матрицу камеры один раз, но и менять её в зависимости от действий пользователя. Например, пользователь может переместить камеру, повернуть камеру в горизонтальной или вертикальной плоскости или даже приблизить/удалить камеру по отношению к точке наблюдения. Необходимая функциональность должна быть оформлена в виде отдельного класса для работы с камерой, структура которого приведена в одном из следующих разделов.

***Матрица модели.***

Каждая модель, как правило, спроектирована вокруг своей собственной системы координат, которая называется локальной системой координат (системой координат модели, model space). Традиционно считается, что данная система является правосторонней, в которой ось Y направлена вверх.

Для размещения объекта на сцене необходимо указать матрицу модели – матрицу, которая переводит все вершины из локальной системы координат в глобальную систему координат. Для вычисления требуемой матрицы необходимо «разместить» локальную систему координат в определенной точке сцены, выразить единичные вектора локальной системы координат и её центр через координаты глобальной системы координат, после чего необходимая матрица вычисляется в соответствии с рисунком приведенном справа.

Следует отметить, что пример генерирования подобной матрицы был подробно рассмотрен в предыдущем семестре. Единственным отличием является то, что теперь используются не стандартные матрицы OpenGL, а пользовательские переменные, которые будут переданы в uniform-переменные шейдера.




Ниже приведен пример матрицы модели для размещения объекта в точке (1, 0, 0) без поворота. Следует обратить внимание, что для работы с матрицами используется класс mat4 из библиотеки glm. Один из конструкторов данного класса позволяет создать матрицу, передавая в качестве параметров вектора, которые будут записаны в соответствующие столбцы матрицы:

```
// матрица модели
mat4 modelMatrix;
// модель располагается в точке (1,0,0) без поворота
modelMatrix = mat4(
       vec4(1, 0, 0, 0),    // 1-ый столбец: направление оси X
       vec4(0, 1, 0, 0),    // 2-ой столбец: направление оси Y
       vec4(0, 0, 1, 0),    // 3-ий столбец: направление оси Z
       vec4(1, 0, 0, 1));   // 4-ый столбец: позиция объекта (начала координат)
```

***Преобразование координат в вершинном шейдере.***

После того, как необходимые матрицы были сгенерированы, их необходимо передать в шейдер в качестве uniform-переменных. При этом усеченные координаты (то есть позиция вершины) в вершинном шейдере будут вычисляться по следующей формуле:

*gl_Position = projectionMatrix * viewMatrix * modelMatrix * vec4 (vPosition, 1.0);*

Нетрудно заметить, что для преобразования вершин требуется выполнить три операции матричного умножения, в то время как матрицы модели, проекции и камеры являются общими для всех вершин одной модели. В связи с этим, для сокращения количества операций, можно заранее перемножить несколько матриц и передать в шейдер уже готовый результат умножения. В частности, также, как и в фиксированном конвейере, для вычислений в шейдере можно использовать две матрицы: матрицу проекции и матрицу наблюдения модели, полученную как произведение матрицы наблюдения н матрицу модели.

Участок кода для вычисления матрицы наблюдения модели приведен ниже. Следует обратить внимание, что произведение матриц не является коммутативным и важен именно такой порядок перемножения:

```
// вычисляем матрицу наблюдения модели
mat4 modelViewMatrix = viewMatrix * modelMatrix;
```

В этом случае формула преобразования вершин в вершинном шейдере заменяется на следующую, более простую, конструкцию:

gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1.0);

Таким образом, количество операций умножений для каждой вершины сократилось с трех до двух, что дает существенный выигрыш в производительности. Закономерно возникает вопрос, почему бы не перемножить сразу и матрицу проекции, передав в шейдер только одну общую матрицу. Для оптимизации это можно сделать, однако следует учесть, что в результате применения матрицы проекции искажаются пропорции объектов и углы между ними (за счет этого достигается эффект перспективы). В связи с этим, если над вершинами в шейдере требуется выполнять еще какие-либо действия, например, вычисление освещения по методу Фонга, рекомендуется отдельно передавать матрицу наблюдения-модели и матрицу проекции.

Полный текст вершинного шейдер из демонстрационного примера приводится ниже:

```
1  #version 330 core
2
3  uniform mat4 projectionMatrix;
4  uniform mat4 modelViewMatrix;
5
6  in vec3 vPosition;
7
8  void main ()
9  {
10     gl_Position = projectionMatrix * modelViewMatrix * vec4 (vPosition, 1);
11 }
```

Полный текст фрагментного шейдера из демонстрационного примера также приводится ниже:

```
1  #version 330 core
2
3  uniform vec4 color;
4
5  out vec4 fragColor;
6
7  void main (void)
8  {
9      fragColor = color;
10 }
```

**Пример функции display для вывода одного объекта на экран.**

Для облегчения ознакомления с принципами формирования матриц проекции, наблюдения и модели, а также с механизмами их передачи в шейдер, ниже приводится полный текст функции display, который можно использовать на начальном этапе для того, чтобы убедиться в правильности понимания этих принципов. Для проверки кода следует использовать функцию drawBox() для вывода объекта, и тексты шейдеров, приведенных выше, для его обработки:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
        // очистка буфера кадра
        glClearColor (1.0, 1.0, 1.0, 1.0);
        glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

        // включение теста глубины (на всякий случай)
        glEnable(GL_DEPTH_TEST);

        // вывод полигонов в виде линий с отсечением нелицевых граней
        glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
        glEnable(GL_CULL_FACE);
        glCullFace(GL_BACK);
```

```
        // формируем матрицу проекции
        mat4 projectionMatrix;
        projectionMatrix = perspective(radians(35.0), 800.0 / 600.0, 1.0, 100.0);

        // генерирование матрицы камеры
        mat4 viewMatrix;
        // позиция камеры - (0, 3, 5)
        vec3 eye = vec3(0.0, 3.0, 5.0);
        // точка, в которую направлена камера - (0, 0, 0);
        vec3 center = vec3(0, 0, 0);
        // примерный вектор "вверх" (0, 1, 0)
        vec3 up = vec3(0, 1, 0);
        // матрица камеры
        viewMatrix = lookAt(eye, center, up);

        // матрица модели
        mat4 modelMatrix;
        // модель располагается в точке (1,0,0) без поворота
        modelMatrix = mat4(
               vec4(1, 0, 0, 0),    // 1-ый столбец: направление оси X
               vec4(0, 1, 0, 0),    // 2-ой столбец: направление оси Y
               vec4(0, 0, 1, 0),    // 3-ий столбец: направление оси Z
               vec4(1, 0, 0, 1));   // 4-ый столбец: позиция объекта (начала координат)

        // вычисляем матрицу наблюдения модели
        mat4 modelViewMatrix = viewMatrix * modelMatrix;

        // активация шейдера
        shader.activate();

        // устанавливаем uniform-переменные для матриц проекции и наблюдения модели
        shader.setUniform("projectionMatrix", projectionMatrix);
        shader.setUniform("modelViewMatrix", modelViewMatrix);

        // устанавливаем значение uniform-переменной для цвета каждого фрагмента
        vec4 color = vec4(0, 0, 1, 1);
        shader.setUniform("color", color);

        // выводим кубик
        drawBox();

        // смена переднего и заднего буферов
        glutSwapBuffers();

        // вычисление FPS и его вывод в заголовок окна
        char windowTitle[128];
        int FPS = getFps();
        sprintf_s(windowTitle, 128, "Laba_03 [%i FPS]", FPS);
        glutSetWindowTitle(windowTitle);

}
```

# Реализация класса для представления камеры.

В рамках данной лабораторной работы требуется реализовать класс для удобного представления и взаимодействия с направленной камерой. Класс предназначен для решения двух основных задач:

1. Хранения параметров камеры и соответствующих матриц. Класс должен иметь методы для удобного задания матриц проекции и наблюдения, а также методы для получения соответствующих матриц для их последующей передачи в шейдер.
2. Управления камерой. Класс должен иметь методы, которые позволяют в удобной форме управлять направленной камерой. В частности, необходимо: передвигать камеру в горизонтальной плоскости, поворачивать её вправо/влево и вверх/вниз, а также приближать и удалять камеру по отношению к точке наблюдения. При изменении позиции камеры матрица наблюдения должна быть пересчитана.

В лабораторной работе нужно реализовать не только вывод кубиков с учетом способа проецирования и позиции наблюдателя, но и перемещение камеры по командам пользователя. В частности, к камере предъявляются следующие требования:

1. Камера должна быть направленной, то есть у камеры есть текущая позиция и точка, в которую она смотрит. Точка наблюдения не обязательно совпадает с началом системы координат, то есть не обязательно равна (0, 0, 0) и по ходу работы программы может меняться.
2. По нажатию на клавиши управления курсором (стрелки вверх/вниз/влево/вправо) происходит передвижение камеры в горизонтальной плоскости. Должна изменяется как точка наблюдения, так и позиция камеры. Передвижение происходит вперед/назад или вправо/влево с учетом текущей ориентации камеры «естественным» способом. Например, движение вправо происходит не строго вдоль вектора oX, а в том направлении, которое указывает вправо при текущей ориентации камеры.

   Передвижения камеры осуществляются с постоянной скоростью, не зависящей от производительности компьютера и частоты вызова функции симуляции, то есть, при передвижении камеры необходимо учитывать время симуляции (время, прошедшее с момента последнего вызова функции симуляции).
3. Вращение камеры происходит вокруг точки наблюдения при зажатой правой кнопке мыши и перемещении курсора. В данном случае скорость поворота определяется не временем симуляции, а тем, насколько сильно изменилось положение курсора мышки с момента последнего шага моделирования. Состояние клавиш, в том числе и состояние правой кнопки мыши, можно получить, используя функцию GetAsyncKeyState, а текущую позицию курсора мышки - с помощью функции GetCursorPos из библиотеки WinAPI. При вращении в вертикальной плоскости необходимо предусмотреть предельные углы: от 5 до 85 градусов.
4. Приближение/удаление камеры должно происходить при вращении колесика мышки. Для определения момента вращения колесика мышки следует воспользоваться механизмом функций обратного вызова библиотеки glut. В частности, с помощью функции glutMouseWheelFunc можно зарегистрировать функцию, которая будет вызываться автоматически при вращении колесика мышки.

Для лучшего понимания того, как должна функционировать камера, рекомендуется ознакомится с демонстрационным примером к данным методическим указаниям. Особое внимание следует уделить перемещению камеры вперед/назад и вправо/влево, поскольку данные перемещения не были реализованы в предыдущем проекте.

Как и любую другую, относительно независимую сущность, требующую обработки, камеру необходимо организовать как отдельный класс и поместить в собственный модуль. Ниже приведен рекомендуемый состав класса, однако разрешается вносить любые оправданные изменения в его структуру. При этом в классе умышленно не приводятся приватные поля, необходимые для его работы, а приводятся только интерфейсные функции. Поля класса, такие как, например, текущие значения матриц проекции и наблюдения, текущая позиция камеры, точка наблюдения и прочие параметры требуется определить самостоятельно.

```
// КЛАСС ДЛЯ РАБОТЫ С КАМЕРОЙ
class Camera
{
public:
	// конструктор по умолчанию
	Camera();

	// установить матрицу проекции
	void setProjectionMatrix(float fovy, float aspect, float zNear, float zFar);
	// получить матрицу проекции
	glm::mat4& getProjectionMatrix();

	// получить матрицу вида
	glm::mat4& getViewMatrix();

	// передвинуть камеру и точку наблюдения в горизонтальной плоскости (OXZ)
	void moveOXZ(float dx, float dz);
	// повернуть в горизонтальной и вертикальной плоскости (угол задается в градусах)
	void rotate(float horizAngle, float vertAngle);
	// приблизить/удалить камеру к/от точки наблюдения
	void zoom(float dR);

private:
	// пересчитать матрицу вида
	void recalculateViewMatrix();

private:
	// необходимые приватные поля класса
	// ...
};
```

Следует также обратить внимание на отсутствие метода apply, который использовался в предыдущем проекте для передачи параметров камеры в OpenGL. Это связанно с тем, что при использовании шейдеров не существует такого понятия как текущая матрица проекции или матрица наблюдения модели, а указанные параметры должны передаваться в каждый используемый шейдер.

# Реализация класса для представления графического объекта.

В лабораторной работе требуется также организовать вывод нескольких графических объектов на экран. Каждый графический объект на данный момент представлен такими параметрами, как цвет и матрица модели, которая определяет позицию и ориентацию графического объекта. Для оперирования графическими объектами необходимо реализовать класс, представленный ниже:

```
// КЛАСС ДЛЯ РАБОТЫ С ГРАФИЧЕСКИМ ОБЪЕКТОМ
class GraphicObject
{
public:
	// конструктор по умолчанию
	GraphicObject();

	// установить цвет объекта
	void setColor(glm::vec4 &color);
	// установить позицию объекта
	void setPosition(glm::vec3 &position);
	// установить угол поворота в градусах относительно оси OY по часовой стрелке
	void setAngle(float degree);

	// получить различные параметры
	glm::vec4& getColor();
	glm::mat4& getModelMatrix();

private:
	// цвет объекта
	glm::vec4 color;
	// матрица модели (задает позицию и ориентацию)
	glm::mat4 modelMatrix;

};
```

Следует обратить внимание на то, что класс не имеет метода для собственной загрузки (например, на основании параметров, заданных во внешнем файле) или метода для вывода на экран, поскольку оба этих метода являются достаточно сложными и не относятся к его функционалу. В частности, в следующих лабораторных работах будут реализованы отдельные классы, ответственные за инициализацию графических объектов и за их вывод на экран. Данный класс является скорее контейнером, объединяющим все данные, необходимые для представления графического объекта, в одном месте.

# Организация программы и функции обратного вызова.

В лабораторной работе №3 необходимо вывести на экран несколько графических объект и реализовать возможность передвижений камеры. Для решения поставленных задач, необходимо объявить общие глобальные данные, которые могут выглядеть следующим образом:

```
// ИСПОЛЬЗУЕМАЯ КАМЕРЫ
Camera camera;
// ИСПОЛЬЗУЕМЫЙ ШЕЙДЕР (ПОКА ТОЛЬКО ОДИН)
Shader shader;
// СПИСОК ГРАФИЧЕСКИХ ОБЪЕКТОВ ДЛЯ ВЫВОДА НА ЭКРАН
vector<GraphicObject> graphicObjects;
```

Для взаимодействия с пользователем в данной серии лабораторных работ используется система функций обратного вызова (callback functions). Функции обратного вызова — это функции, которые регистрируются в системе (в данном случае в библиотеке glut) и которые будут вызваны автоматически при наступлении какого-либо события для обработки этого события. Рассмотрим основные функции обратного вызова и их применение.

**Функция simulation.**

Функция simulation регистрируется с помощью функции glutIdleFunc и будет вызываться всякий раз при простое процессора, то есть максимально часто. В функции необходимо определить время симуляции (время, прошедшее с момента последнего вызова данной функции) которое будет учитываться при передвижении камеры. Кроме того, в данной функции необходимо определить, требуется ли передвигать или поворачивать камеру. Для этого используются ранее упоминавшиеся функции GetAsyncKeyState и GetCursorPos из библиотеки WinAPI. Общая структура функции выглядит следующим образом:

```
// функция вызывается когда процессор простаивает, т.е. максимально часто
void simulation()
{
	// определяем время симуляции
	float simulationTime = ...
	// определяем необходимость передвижения камеры
	...
	camera.move(dx, dz);
	// определяем необходимость вращения камеры
	...
	camera.rotate(rx, ry);
	// вызываем принудительную перерисовку окна
	glutPostRedisplay();

};
```

**Функция reshape.**

Функция Reshape регистрируется с помощью функции glutReshapeFunc и будет вызываться автоматически каждый раз при изменении размеров окна. Функция вызывается, в том числе, и при создании окна. Основное назначение функции Reshape – это изменение порта просмотра и матрицы проекции с учетом новых размеров окна. Полный код функции может выглядеть следующим образом:

```
// функция, вызываемая при изменении размеров окна
void reshape(int w,int h)
{
	// установить новую область просмотра, равную всей области окна
	glViewport(0, 0, w, h);
	// устанавливаем матрицу проекции
	camera.setProjectionMatrix(35.0f, (float)w / h, 1.0f, 500.0f);

}
```

**Функция display.**

Данная функция регистрируется с помощью функции glutDisplayFunc и вызывается всякий раз, когда требуется перерисовать окно, в том числе при создании окна, изменении его размеров или после использования функции glutPostRedisplay. Функция должна отчистить окно, вывести необходимые объекты, используя определенные шейдеры, камеры и прочие параметры, после чего поменять местами передний и задний буфер. При использовании вышеуказанных глобальных объектов текст функции может выглядеть так, как показано ниже:

```
// функция вызывается при перерисовке окна
// в том числе принудительно, по команде glutPostRedisplay
void display()
{
	// очистка буфера кадра
	glClearColor (1.0, 1.0, 1.0, 1.0);
	glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT );

	// включение теста глубины (на всякий случай)
	glEnable(GL_DEPTH_TEST);

	// вывод полигонов в виде линий с отсечением нелицевых граней
	glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
	glEnable(GL_CULL_FACE);
	glCullFace(GL_BACK);

	// активируем шейдер, используемый для вывода объекта
	shader.activate();

	// устанавливаем матрицу проекции
	mat4& projectionMatrix = camera.getProjectionMatrix();
	shader.setUniform("projectionMatrix", projectionMatrix);

	// получаем матрицу камеры
	mat4& viewMatrix = camera.getViewMatrix();

	// выводим все объекты
	for (auto& grObj : graphicObjects) {

		// устанавливаем матрицу наблюдения модели
		mat4 modelViewMatrix = viewMatrix * grObj.getModelMatrix();
		shader.setUniform("modelViewMatrix", modelViewMatrix);

		// устанавливаем цвет
		shader.setUniform("color", grObj.getColor());

		// выводим модель кубика
		drawBox();
	}

	// меняем передний и задний буферы цвета
	glutSwapBuffers();

	// вычисление FPS и его вывод в заголовок окна
	char windowTitle[128];
	int FPS = getFps();
	sprintf_s(windowTitle, 128, "Laba_03 [%i FPS]", FPS);
	glutSetWindowTitle(windowTitle);

}
```

Следует отметить, что шейдеры замещают собой только некоторые ступени конвейера, а не весь конвейер OpenGL. В частности, как видно из приведенного примера, по-прежнему работают такие режимы OpenGL как отсечение лицевых/нелицевых граней и режим вывода полигонов (сплошная заливка или в виде линий).

**Функция mouseWheel.**

Последняя функция обратного вызова, используемая в данной лабораторной работе, ранее не использовалась и необходима для отслеживания вращения колесика мышки. Данная функция регистрируется с помощью функции glutMouseWheelFunc.

Общая структура функции приведена ниже:

```
// функция обработки вращения колесика мышки
void mouseWheel(int wheel, int direction, int x, int y)
{
	// определяем, на сколько необходимо приблизить/удалить камеру
	float delta = ...;
	camera.zoom(delta);
}
```

Дополнительную информацию по это функции, а также по другим функциям freeglut, можно найти, например, по следующей ссылке:

http://freeglut.sourceforge.net/docs/api.php

# Задание к лабораторной работе.

Лабораторная работе №3 строится на основе предыдущей работы. К программе предъявляются следующие требования:

1. Необходимо реализовать класс GraphicObject для представления графических объектов в соответствии с описанием, приведенным выше.
2. Необходимо также реализовать класс Camera для работы с направленной камерой. Структура класса приведена в соответствующем разделе.
3. Используя реализованные классы необходимо вывести несколько графических объектов на экран. Расположение графических объектов требуется задать самостоятельно и не обязательно так, как в демонстрационном примере.
4. Необходимо организовать передвижение камеры в соответствии с описанием, приведенным в соответствующем разделе, и в соответствии с демонстрационным примером. Особое внимание следует уделить трем ключевым моментам:
    - Вращение камеры происходит при зажатой правой кнопке мыши. При этом угол поворота определяется не временем симуляции, а тем, на какой расстояние был перемещен курсор мыши с момента последнего шага симуляции.
    - Приближение/удаление камеры к точке наблюдения осуществляется при вращении колесика мыши, для чего необходимо задействовать одну из функций обратного вызова.
    - Передвижение вперед/назад и влево/вправо должно происходить с учетом текущей ориентации камеры (её угла поворота), а не вдоль координатных осей! Кроме того, следует учесть, что передвижение, в отличие от поворота, происходит при нажатии кнопок клавиатуры и зависит от времени симуляции.

Отчет к лабораторной работе должен быть сформирован на основе шаблона, представленного на следующей странице. При оформлении отчета необходимо внести свою фамилию, имя, отчество, номер группы, а также заполнить таблицу. При заполнении таблицы необходимо дать краткое описание реализованным модулям, указать количество строк каждого модуля, а также подсчитать общее количество строк во всем проекте.

# Архитектура графических систем (6 семестр)

**Выполнил: Иванов Иван Иванович**

Группа: 00-00

Проверил: Галибин Сергей Владимирович

## Задание:

В лабораторной работе №3 необходимо организовать вывод нескольких графических объектов и передвижение направленной камеры.

## Защита лабораторной работы:

Для защиты лабораторной работы необходимо знать:

1. **[КГ]** Для чего необходима матрица проекции.
2. **[КГ]** Для чего необходима матрица наблюдения.
3. **[КГ]** Для чего необходима матрица модели.
4. **[КГ]** Как происходит преобразование вершины с учетом этих трех матриц.
5. **[КГ]** Что такое усеченная пирамида видимости.
6. **[CPP]** Что такое функции обратного вызова (callback-функции) в целом, какие callback-функции используются в данной лабораторной работе и при наступлении каких событий они вызываются.
7. **[CPP]** Для чего необходима функция GetAsyncKeyState, какие у неё параметры и формат возвращаемого значения.
8. **[CPP]** Для чего необходима функция GetCursorPos, какие у неё параметры и формат возвращаемого значения.

## Состав проекта:

| **Название модуля и его назначение** | **Количество строк** | |
|---|---|---|
| | ***.h** | ***.cpp** |
| main - основная программа | | |
| Shader - | | |
| Camera - | | |
| GraphicObject - | | |
| | | |

## Приложение 1.

## Процедура инициализации и вывода модели кубика.

В качестве модели в данной лабораторной работе используется кубик, лежащий в центре локальной системы координат и имеющий длину ребер равную единице. Для вывода кубика используется функция drawBox, приведенная ниже. Все необходимые данные объявлены в самой функции как статичные переменные, которые инициализируются в момент первого вызова данной функции:

```
// функция вывода кубика с ребрами единичной длины
// каждая координата (x, y, z) меняется от -0.5 до +0.5
void drawBox()
{
	// переменные для вывода объекта (прямоугольника из двух треугольников)
	static GLuint VAO_Index = 0; // индекс VAO-буфера
	static GLuint VBO_Index = 0; // индекс VBO-буфера
	static int VertexCount = 0;		// количество вершин
	static bool init = true;

	if (init) {
		// создание и заполнение VBO
		glGenBuffers(1, &VBO_Index);
		glBindBuffer(GL_ARRAY_BUFFER, VBO_Index);
		GLfloat Verteces[] = {
			// передняя грань (два треугольника)
			-0.5, +0.5, +0.5, -0.5, -0.5, +0.5, +0.5, +0.5, +0.5,
			+0.5, +0.5, +0.5, -0.5, -0.5, +0.5, +0.5, -0.5, +0.5,
			// задняя грань (два треугольника)
			+0.5, +0.5,   -0.5, +0.5, -0.5, -0.5, -0.5, +0.5, -0.5,
			-0.5, +0.5,   -0.5, +0.5, -0.5, -0.5, -0.5, -0.5, -0.5,
			// правая грань (два треугольника)
			+0.5, -0.5,   +0.5, +0.5, -0.5, -0.5, +0.5, +0.5, +0.5,
			+0.5, +0.5,   +0.5, +0.5, -0.5, -0.5, +0.5, +0.5, -0.5,
			// левая грань (два треугольника)
			-0.5, +0.5,   +0.5, -0.5, +0.5, -0.5, -0.5, -0.5, +0.5,
			-0.5, -0.5,   +0.5, -0.5, +0.5, -0.5, -0.5, -0.5, -0.5,
			// верхняя грань (два треугольника)
			-0.5, +0.5, -0.5, -0.5, +0.5, +0.5, +0.5, +0.5, -0.5,
			+0.5, +0.5, -0.5, -0.5, +0.5, +0.5, +0.5, +0.5, +0.5,
			// нижняя грань (два треугольника)
			-0.5, -0.5, +0.5, -0.5, -0.5, -0.5, +0.5, -0.5, +0.5,
			+0.5, -0.5, +0.5, -0.5, -0.5, -0.5, +0.5, -0.5, -0.5
		};
		glBufferData(GL_ARRAY_BUFFER, sizeof(Verteces), Verteces, GL_STATIC_DRAW);

		// создание VAO
		glGenVertexArrays(1, &VAO_Index);
		glBindVertexArray(VAO_Index);

		// инициализация VAO
		glBindBuffer(GL_ARRAY_BUFFER, VBO_Index);
		int location = 0;
		glVertexAttribPointer(location, 3, GL_FLOAT, GL_FALSE, 0, 0);
		glEnableVertexAttribArray(location);

		// "отвязка" буфера VAO на всякий случай, чтоб случайно не испортить
		glBindVertexArray(0);

		// указание количество вершин
		VertexCount = 6 * 6;
		init = false;
	}

	// вывод модели кубика на экран
	glBindVertexArray(VAO_Index);
	glDrawArrays(GL_TRIANGLES, 0, VertexCount);

}
```